

Derivative Free Optimization

methods, algorithms and implementations

Ahmed Bayoumy

Siemens DISW

October 24, 2024

Overview

- What is blackbox optimization?
- Functions differentiability
- Direct search methods
- Mesh adaptive direct search method
- Multiobjective optimization using MADS
- Implementations

Overview

- What is blackbox optimization?
- Functions differentiability
- Direct search methods
- Mesh adaptive direct search method
- Multiobjective optimization using MADS
- Implementations

Blackbox Optimization Problems

We consider blackbox optimization problems:

$$\min_{x \in \Omega} f(x)$$

where evaluations of f and the functions defining Ω are usually the result of a computer code (a blackbox).



Each call to the simulation is time-consuming

Blackbox Optimization Problems

We consider blackbox optimization problems:

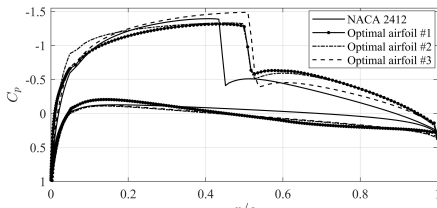
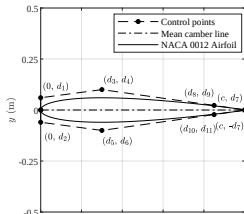
$$\min_{x \in \Omega} f(x)$$

where evaluations of f and the functions defining Ω are usually the result of a computer code (a blackbox).



Each call to the simulation is time-consuming

Transonic airfoil shape optimization using TSFOIL, SU2, OpenFoam, fluent ...etc.



Blackbox Optimization Problems

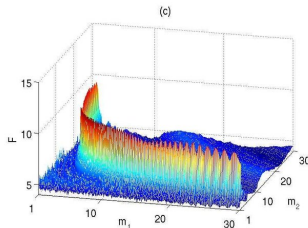
We consider blackbox optimization problems:

$$\min_{x \in \Omega} f(x)$$

where evaluations of f and the functions defining Ω are usually the result of a computer code (a blackbox).



Derivatives cannot be trusted or evaluated.



Blackbox Optimization Problems

We consider blackbox optimization problems:

$$\min_{x \in \Omega} f(x)$$

where evaluations of f and the functions defining Ω are usually the result of a computer code (a blackbox).



BBO challenges



Possible bad properties of a blackbox from J. Simonis, 2009, *The Boeing Company*

Blackbox Optimization Problems

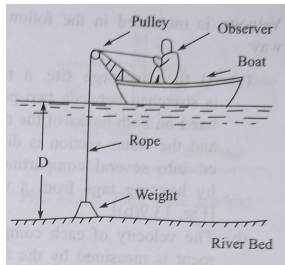
We consider blackbox optimization problems:

$$\min_{x \in \Omega} f(x)$$

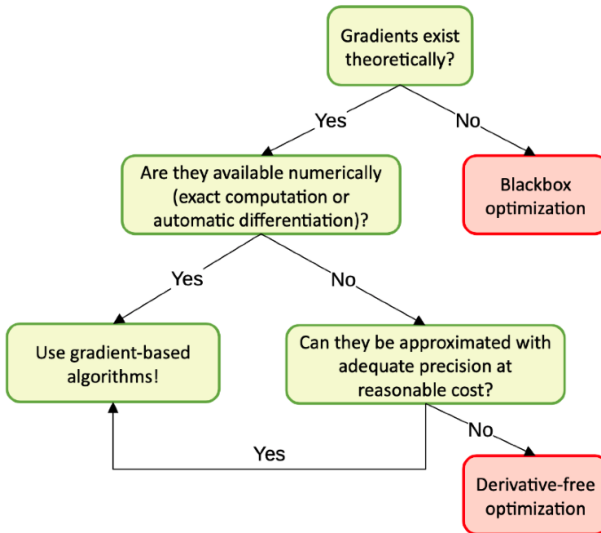
where evaluations of f and the functions defining Ω are usually the result of a computer code (a blackbox).



John Dennis analogy [Powell 1994]



Gradients availability



Blackbox Optimization Problems

Definition: Blackbox optimization

It refers to problems where objective and constraint functions cannot be exploited.

Often the case when their evaluation requires the execution (usually time-consuming) simulation using computational models, typically inaccessible by the user.

Blackbox Optimization Problems

Definition: Blackbox optimization

It refers to problems where objective and constraint functions cannot be exploited.

Often the case when their evaluation requires the execution (usually time-consuming) simulation using computational models, typically inaccessible by the user.

Definition: Derivative-free optimization

It refers to the use of algorithms that utilize only function values because their derivatives are either not defined or not available.

Gradient approximations may sometimes be obtained, but the amount of work required to ensure they are dependable may not be worth the effort.

Types of constraints

The domain: $\Omega = \{x \in X : c_j(x) \leq 0, j \in J\} \subset \mathbb{R}^n$

- The set X represents *unrelaxable constraints*
- $c_j(x) \leq 0$ are *relaxable constraints*
- *Hidden constraints*: when the simulation fails for points in Ω

Properties of a function

- We consider a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$.
- We define some properties of f at x , a point of its domain.
- We say that these properties apply near x if the property is satisfied on some open neighborhood of x .
- We can also consider some properties on a domain $X \subset \mathbb{R}^n$.
- f is continuous at $x \in \mathbb{R}^n$ if the limit $\lim_{y \rightarrow x} f(y)$ exists and is equal to $f(x)$.

Types of variables

The decision variables x can be any combination of

- Continuous $\mathbb{R}$
- Integer $\mathbb{N}$ or $\mathbb{Z}$
- binary $\{0, 1\}$
- granular $\{0, 0.05, 0.10, \dots, 0.95, 1.00\}$
- Categorical $\{\text{ReLU}, \text{tanh}, \dots\}$
 - Ex: Hyper-parameter optimization of deep neural network

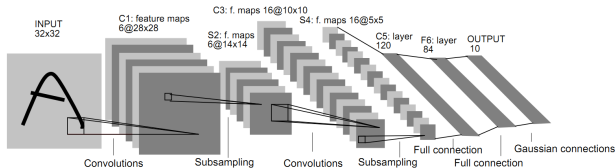


Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

- No ordinal property
- The number of convolution layers impacts the total number of optimization variables

- What is blackbox optimization?
- **Functions differentiability**
- Direct search methods
- Mesh adaptive direct search method
- Multiobjective optimization using MADS
- Implementations

Differentiability

Consider the function $f : \mathbb{R}^n \rightarrow \mathbb{R}$.

- f is **differentiable** at $x \in \mathbb{R}^n$ if there exists $g \in \mathbb{R}^n$ such that

$$\lim_{y \rightarrow x} \frac{f(y) - f(x) - g^\top(y - x)}{\|y - x\|} = 0$$

- If this g exists, it is unique and is called the **gradient** of f at x , denoted $\nabla f(x)$.
- If f is differentiable at x , then f is continuous at x .
- **Partial derivatives:**

$$\nabla f(x) = \left(\frac{\partial f(x)}{\partial x_1}, \frac{\partial f(x)}{\partial x_2}, \dots, \frac{\partial f(x)}{\partial x_n} \right)^\top$$

Differentiability classes

- A function $f: \mathbb{R}^n \rightarrow \mathbb{R}$ is said of class $\mathcal{C}^k$, denoted $f \in \mathcal{C}^k$, with $0 \leq k \leq \infty$, if all the possible partial derivatives of the form

$$\frac{\partial^k f}{\partial x_{i_1} \partial x_{i_2} \cdots \partial x_{i_k}}$$

exist and are continuous, where $i_\ell \in \{1, 2, \dots, n\}$ for all $\ell \in \{1, 2, \dots, k\}$.

- $\mathcal{C}^0$: Continuous functions.
- $\mathcal{C}^1$: Continuously differentiable functions.
- $\mathcal{C}^\infty$: Smooth functions.

Lipschitz functions

- f is **Lipschitz** on the set $\mathcal{X} \subset \mathbb{R}^n$ if there exists a scalar $K > 0$ such that

$$|f(x) - f(y)| \leq K \|x - y\| \text{ for all } x, y \in \mathcal{X}$$

- K is called the **Lipschitz constant**.

- Examples of non-Lipschitz functions:

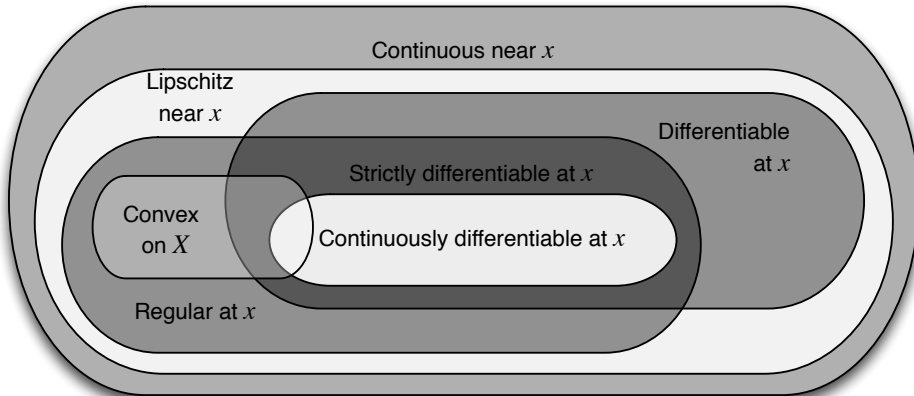
- $f(x) = \sqrt{x}$ for $x \geq 0$.
- Discontinuous functions, $\tan(x)$ for $x \in (-\pi/2, \pi/2)$, $\frac{1}{x}$ for $x \in \mathbb{R}$.
- $f(x) = x^2$ for $x \in \mathbb{R}$.

Examples of Lipschitz functions:

- $f(x) = \sqrt{x^2 + 5}$ for $x \in \mathbb{R}$, $K = 1$.
- $\sin(x)$ for $x \in \mathbb{R}$, $K = 1$.
- $f(x) = |x|$ for $x \in \mathbb{R}$, $K = 1$.

Summary of function types

$f : X \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$ is...



Convergence analysis (1/2)

- An optimization algorithm is not considered **heuristic** when it is backed by a **convergence analysis** which ensures some properties at the resulting solution $\hat{x}$.
- This analysis typically depends on some assumptions made about the nature of the problem. For example: differentiability of f , convexity of Ω , etc.
- Usually, these properties are given as **necessary** or **sufficient optimality conditions**.
- Recall that **global convergence** refers to independence of the starting point.

Convergence analysis (2/2)

- In DFO, we expect global convergence to solutions satisfying some local and necessary optimality conditions, when the function is supposed Lipschitz.
- However, a blackbox has no exploitable property and cannot be proven Lipschitz.
- **But** consider the following choice between two algorithms to apply to such a problem:
 - Algorithm $\mathcal{A}$ is a heuristic; it **may** yield a point $\hat{x}$ where $\nabla f(\hat{x}) \neq 0$ when f is differentiable.
 - Algorithm $\mathcal{B}$ guarantees $\nabla f(\hat{x}) = 0$ when f is differentiable.

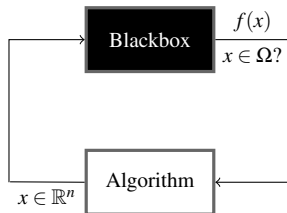
The choice is obvious.

- What is blackbox optimization?
- Functions differentiability
- **Direct search methods**
- Mesh adaptive direct search method
- Multiobjective optimization using MADS
- Implementations

General scheme

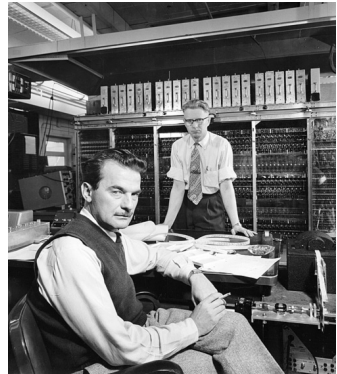
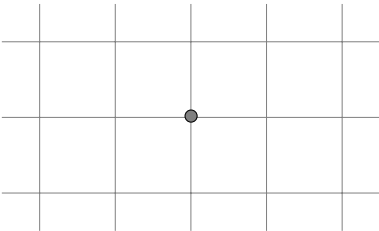
Direct search methods launch the blackbox simulation at tentative trial points in hopes of improving the current best solution.

* The way that the trial points are generated defines the method



Unconstrained optimization - Coordinate search

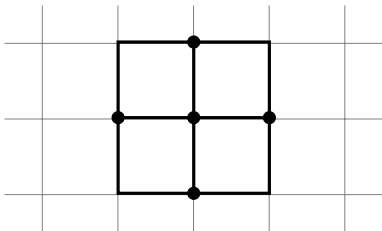
1952 Fermi and Metropolis
Coordinate Search algorithm.



Metropolis and Richardson in front of the MANIAC computer
Source <http://www.ominous-valve.com/maniac.html>

Unconstrained optimization - Coordinate search

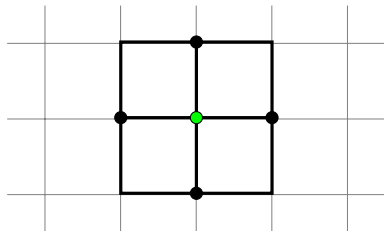
1952 Fermi and Metropolis
Coordinate Search algorithm.



- **INITIALIZATION:**
 x_0 : starting point in $\mathbb{R}^n$
 $\Delta_0 > 0$: initial step size
- **POLL STEP:** for $k = 0, 1, \dots$
if $f(t) < f(x_k)$ for $t \in P_k := \{x_k \pm \Delta_k e_i : i = 1, 2, \dots, n\}$:
 $x_{k+1} \leftarrow t$
 $\Delta_{k+1} \leftarrow \Delta_k$
else (failure): x_k is a local minimum relatively to P_k :
 $x_{k+1} \leftarrow x_k$
 $\Delta_{k+1} \leftarrow \Delta_k/2$

Unconstrained optimization - Coordinate search

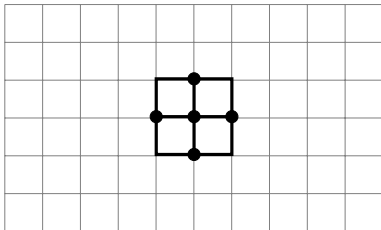
1952 Fermi and Metropolis
Coordinate Search algorithm.



- **INITIALIZATION:**
 x_0 : starting point in $\mathbb{R}^n$
 $\Delta_0 > 0$: initial step size
- **POLL STEP:** for $k = 0, 1, \dots$
if $f(t) < f(x_k)$ for $t \in P_k := \{x_k \pm \Delta_k e_i : i = 1, 2, \dots, n\}$:
 $x_{k+1} \leftarrow t$
 $\Delta_{k+1} \leftarrow \Delta_k$
else (failure): x_k is a local minimum relatively to P_k :
 $x_{k+1} \leftarrow x_k$
 $\Delta_{k+1} \leftarrow \Delta_k/2$

Unconstrained optimization - Coordinate search

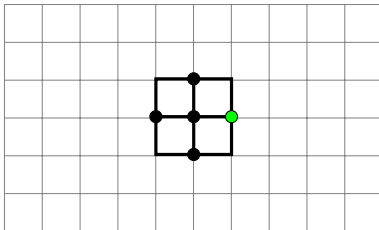
1952 Fermi and Metropolis
Coordinate Search algorithm.



- **INITIALIZATION:**
 x_0 : starting point in $\mathbb{R}^n$
 $\Delta_0 > 0$: initial step size
- **POLL STEP:** for $k = 0, 1, \dots$
if $f(t) < f(x_k)$ for $t \in P_k := \{x_k \pm \Delta_k e_i : i = 1, 2, \dots, n\}$:
 $x_{k+1} \leftarrow t$
 $\Delta_{k+1} \leftarrow \Delta_k$
else (failure): x_k is a local minimum relatively to P_k :
 $x_{k+1} \leftarrow x_k$
 $\Delta_{k+1} \leftarrow \Delta_k/2$

Unconstrained optimization - Coordinate search

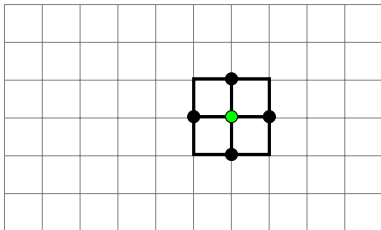
1952 Fermi and Metropolis
Coordinate Search algorithm.



- **INITIALIZATION:**
 x_0 : starting point in $\mathbb{R}^n$
 $\Delta_0 > 0$: initial step size
- **POLL STEP:** for $k = 0, 1, \dots$
if $f(t) < f(x_k)$ for $t \in P_k := \{x_k \pm \Delta_k e_i : i = 1, 2, \dots, n\}$:
 $x_{k+1} \leftarrow t$
 $\Delta_{k+1} \leftarrow \Delta_k$
else (failure): x_k is a local minimum relatively to P_k :
 $x_{k+1} \leftarrow x_k$
 $\Delta_{k+1} \leftarrow \Delta_k / 2$

Unconstrained optimization - Coordinate search

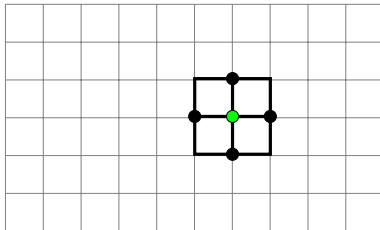
1952 Fermi and Metropolis
Coordinate Search algorithm.



- **INITIALIZATION:**
 x_0 : starting point in $\mathbb{R}^n$
 $\Delta_0 > 0$: initial step size
- **POLL STEP:** for $k = 0, 1, \dots$
if $f(t) < f(x_k)$ for $t \in P_k := \{x_k \pm \Delta_k e_i : i = 1, 2, \dots, n\}$:
 $x_{k+1} \leftarrow t$
 $\Delta_{k+1} \leftarrow \Delta_k$
else (failure): x_k is a local minimum relatively to P_k :
 $x_{k+1} \leftarrow x_k$
 $\Delta_{k+1} \leftarrow \Delta_k/2$

Unconstrained optimization - Coordinate search

1952 Fermi and Metropolis
Coordinate Search algorithm.



- Remarks

- Evaluation of x_0 is counted
- Extension with bounds: if the algorithm generates a trial point outside of the boundary, then do not evaluate and consider as failure
- If the algorithm generates a trial point x found in the cache, then do not evaluate and consider as failure

- INITIALIZATION:

x_0 : starting point in $\mathbb{R}^n$

$\Delta_0 > 0$: initial step size

- POLL STEP: for $k = 0, 1, \dots$

if $f(t) < f(x_k)$ for $t \in P_k := \{x_k \pm \Delta_k e_i : i = 1, 2, \dots, n\}$:

$$x_{k+1} \leftarrow t$$

$$\Delta_{k+1} \leftarrow \Delta_k$$

else (failure): x_k is a local minimum relatively to P_k :

$$x_{k+1} \leftarrow x_k$$

$$\Delta_{k+1} \leftarrow \Delta_k / 2$$

CS variants

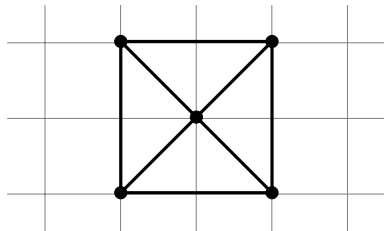
Complete			Opportunist			Ordered		
k	trial point $t^\dagger$	$f(t)$	k	trial point $t^\dagger$	$f(t)$	k	trial point $t^\dagger$	$f(t)$
0	[3, 2]	29286	0	[3, 2]	29286	0	[3, 2]	29286
	[2, 3]	4772		[2, 3]	4772		[2, 3]	4772
	$x^1 = [1, 2]$	166		$x^1 = [1, 2]$	166		$x^1 = [1, 2]$	166
	[2, 1]	4176						
1	[2, 2]	4401	1	[2, 2]	4401	1	$x^2 = [0, 2]$	81
	[1, 3]	262		[1, 3]	262		[−1, 2]	2646
	$x^2 = [0, 2]$	81		$x^2 = [0, 2]$	81		[0, 3]	152
	[1, 1]	106	2	[1, 2]	166	[1, 2]	166	
				[0, 3]	152	$x^3 = [0, 1]$	36	
2	[1, 2]	166		[−1, 2]	2646	3	$x^4 = [0, 0]$	17
	[0, 3]	152	$x^3 = [0, 1]$	36				
	[−1, 2]	2646	3	[1, 1]	106	4	[0, −1]	24
	$x^3 = [0, 1]$	36		[0, 2]	81	[0, 1]	36	
				[−1, 1]	2466	[1, 0]	82	
3	[1, 1]	106		$x^4 = [0, 0]$	17		[−1, 0]	2402
	[0, 2]	81	4	[1, 0]	82	5	[0, −0.5]	17.25
	[−1, 1]	2466		[0, 1]	36		[0, 0.5]	23.25
	$x^4 = [0, 0]$	17					$x^6 = [0.5, 0]$	1.0625
				[−1, 0]	2402	6	[1, 0]	82
		[0, −1]		24	$x^7 = [0.5, −0.5]$		0.375	
4	[1, 0]	82		$x^6 = [0.5, 0]$	1.0625	7	[0.5, −1]	4.3125
	[0, 1]	36			[1, −0.5]		83.5	
Incumbent solutions after a total of 100 function evaluations								
[0.398, −0.332] 2.2×10^{-5}			[0.3999, −0.3330] 9.8×10^{-7}			[0.4000, −0.3334] 1.7×10^{-8}		

CS evolution

1961 Hooke and Jeeves - Pattern Search

1997 Torczon - Generalized Pattern Search

Mesh coarsens on successful iterations

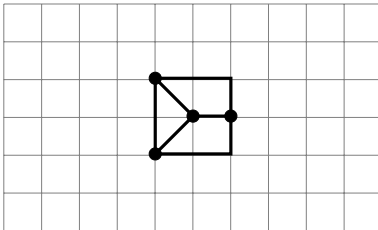


CS evolution

1961 Hooke and Jeeves - Pattern Search

1997 Torczon - Generalized Pattern Search

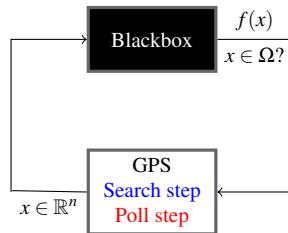
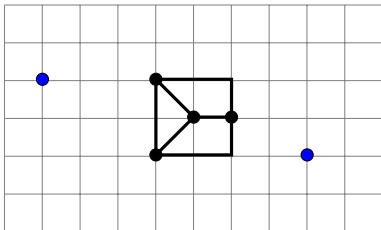
Mesh refines on unsuccessful iterations



CS evolution

1961 Hooke and Jeeves - Pattern Search

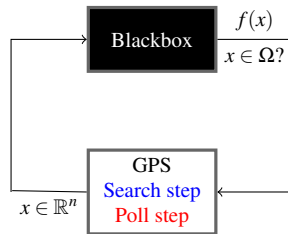
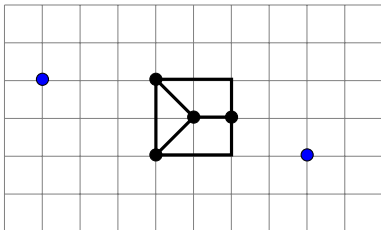
1997 Torczon - Generalized Pattern Search



CS evolution

1961 Hooke and Jeeves - Pattern Search

1997 Torczon - Generalized Pattern Search



Theorem (Convergence analysis 2003 - Clark calculus)

Let $\hat{x}$ be an accumulation point of the sequence of mesh local optimizers on meshes that get infinitely fine.

If f is Lipschitz near $\hat{x}$, then $f^\circ(\hat{x}; d) \geq 0$ for all directions d used infinitely often.

The set of such directions forms a positive basis for $\mathbb{R}^n$

[0] Initializations (x_0, Δ_0)**[1] Iteration k** **[1.1] (global) Search**

select a finite number of mesh points
sort these points
evaluate candidates opportunistically

[1.2] (local) Poll (if the Search failed)

construct poll set $P_k = \{x_k + \Delta_k d : d \in D_k\}$
sort(P_k)
evaluate candidates opportunistically

[2] Updates

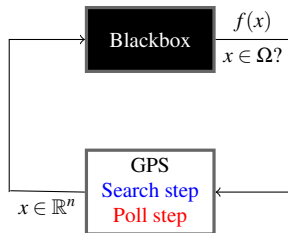
if success

$x_{k+1} \leftarrow$ success point
possibly increase Δ_k

else

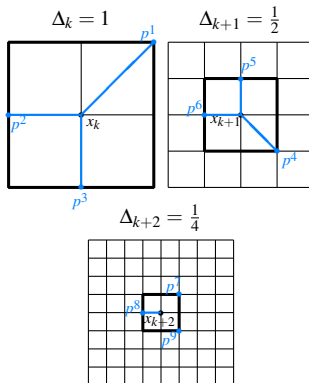
$x_{k+1} \leftarrow x_k$
decrease Δ_k

$k \leftarrow k + 1$, stop or go to **[1]**



GPS: Example of poll directions

$P_k = \{x_k + \Delta_k d : d \in D_k\}$; $n + 1$ mesh points at distance Δ_k from x_k



14 different ways of defining D_k on this mesh

- What is blackbox optimization?
- Functions differentiability
- Direct search methods
- **Mesh adaptive direct search method**
- Multiobjective optimization using MADS
- Implementations

The MADS algorithm

MADS main algorithm

1: Initialization

$x^0 \in \mathbb{R}^n$ initial point
 $\Delta^0 \in (0, \infty)$ initial frame size parameter

2: Parameter Update

set the mesh size parameter to $\delta^k \leq \Delta^k$
define the mesh M^k

3: “Optional” global [Search](#) on mesh M^k

if $f(t) < f(x^k)$ for some t in a finite subset of the mesh M^k
set $x^{k+1} \leftarrow t$ and $\Delta^{k+1} \leftarrow 2\Delta^k$ and go to Step 5.

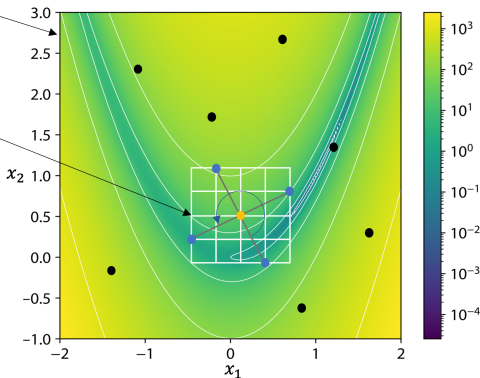
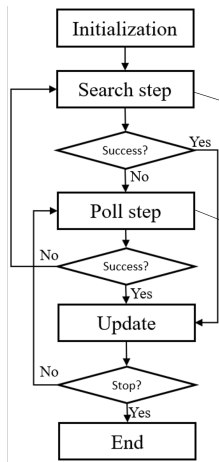
4: Local [Poll](#) on Mesh M^k

select a positive spanning set D_Δ^k
if $f(t) < f(x^k)$ for some $t \in P^k = \{x^k + \delta^k d : d \in D_\Delta^k\}$ in a finite subset of the mesh M^k
set $x^{k+1} \leftarrow t$ and $\Delta^{k+1} \leftarrow 2\Delta^k$
else set $x^{k+1} \leftarrow x^k$ and $\Delta^{k+1} \leftarrow \frac{1}{2}\Delta^k$

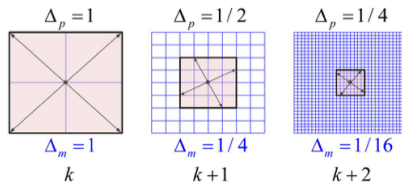
5: Termination check

otherwise $k \leftarrow k + 1$ and return to Step 2.

MADS algorithm workflow



Mesh and frame definitions



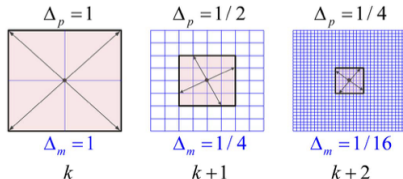
Definition: Mesh and Mesh Size Parameter

Let $G \in \mathbb{R}^{n \times n}$ be an invertible matrix and the columns of $Z \in \mathbb{Z}^{n \times p}$ form a positive spanning set for $\mathbb{R}^n$. Define $D = GZ$. The mesh of coarseness $\delta^k > 0$ generated by D , centered at the incumbent solution $x_k \in \mathbb{R}^n$, is defined by $M^k := \{x^k + \delta^k D_y : y \in \mathbb{N}^p \subset \mathbb{R}^n, \}$ where δ^k is called the mesh size parameter.

Definition: Frame and Frame Size Parameter

Let $G \in \mathbb{R}^{n \times n}$ be an invertible matrix and the columns of $Z \in \mathbb{Z}^{n \times p}$ form a positive spanning set for $\mathbb{R}^n$. Define $D = GZ$. Select a mesh size parameter $\delta^k > 0$ and let Δ^k be such that $\delta^k \leq \Delta^k$. The frame of extent Δ^k generated by D , centered at the incumbent solution $x_k \in \mathbb{R}^n$, is defined by $F^k := \{x \in M^k : \|x - x^k\|_\infty \leq \Delta^k b\}$ with $b = \max\{\|d'\|_\infty : d' \in \mathbb{D}\}$ and Δ^k is called the frame size parameter.

Dense (rich) set of polling directions



Definition: Asymptotically Dense

The set $V \subseteq \mathbb{R}^n$ is said to be asymptotically dense if the normalised set $\{v/||v|| : v \in V\}$ is dense on the *unit sphere* $S = \{w \in \mathbb{R}^n : |w| = 1\}$.

Definition: Householder Matrix

Let $V \subseteq \mathbb{R}^n$ be a *normalized vector*. The Householder matrix H associated with v is $H := I - 2vv^T \in \mathbb{R}^{n \times n}$, where I is the $n \times n$ *identity matrix*.

- What is blackbox optimization?
- Functions differentiability
- Direct search methods
- Mesh adaptive direct search method
- **Multiobjective optimization using MADS**
- Implementations

Multiobjective optimization problem

- The **multiobjective optimization problem (MOP)** can be formally stated as

$$\min_{x \in \Omega} F(x)$$

- where

$$F : \Omega \rightarrow \{\mathbb{R} \cup \{+\infty\}\}^p$$

and

$$F(x) = (f^{(1)}(x), f^{(2)}(x), \dots, f^{(p)}(x))$$

- p is the number of objective functions
- Case $p = 2$: **Biobjective optimization problem (BOP)**
- The feasible set Ω remains unchanged
- Typically, the different objectives are **contradictory**: A decrease in one objective causes an increase in the other objectives

Trade-off challenge

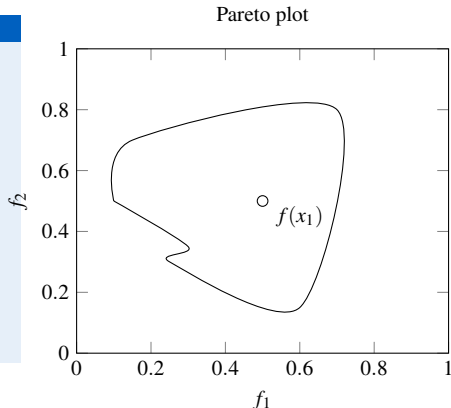
- The **Single-objective optimization** Particular case where $p = 1$. An optimal solution typically consists of one single vector $x \in \Omega$
- Multiobjective optimization: There is usually no such vector that simultaneously minimizes all of the $p \leq 2$ objective functions
- The solution consists of a set of **trade-off solutions** in Ω , the **Pareto solutions**
- The methods presented in this lesson construct an approximation to this set

Pareto notion

Definition: Pareto dominance

Given two decision vectors x_1 and x_2 in the feasible decision set, it is said that:

- $x_1 \leq x_2$ (x_1 **weakly dominates** x_2) iff $\forall i = 1, 2, \dots, m, f_i(x_1) \leq f_i(x_2)$
- $x_1 < x_2$ (x_1 **dominates** x_2) iff $\forall i = 1, 2, \dots, m, f_i(x_1) \leq f_i(x_2)$ and it exists at least an index i_0 such that $f_{i_0}(x_1) < f_{i_0}(x_2)$
- $x_1 \sim x_2$ (x_1 and x_2 are **indifferent**) if x_1 does not dominate x_2 nor x_2 does not dominate x_1

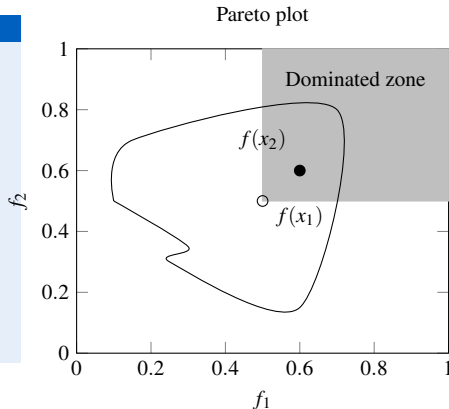


Pareto notion

Definition: Pareto dominance

Given two decision vectors x_1 and x_2 in the feasible decision set, it is said that:

- $x_1 \leq x_2$ (x_1 **weakly dominates** x_2) iff $\forall i = 1, 2, \dots, m, f_i(x_1) \leq f_i(x_2)$
- $x_1 < x_2$ (x_1 **dominates** x_2) iff $\forall i = 1, 2, \dots, m, f_i(x_1) \leq f_i(x_2)$ and it exists at least an index i_0 such that $f_{i_0}(x_1) < f_{i_0}(x_2)$
- $x_1 \sim x_2$ (x_1 and x_2 are **indifferent**) if x_1 does not dominate x_2 nor x_2 does not dominate x_1

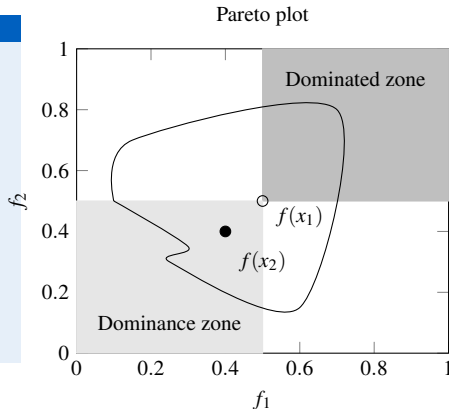


Pareto notion

Definition: Pareto dominance

Given two decision vectors x_1 and x_2 in the feasible decision set, it is said that:

- $x_1 \leq x_2$ (x_1 **weakly dominates** x_2) iff $\forall i = 1, 2, \dots, m, f_i(x_1) \leq f_i(x_2)$
- $x_1 < x_2$ (x_1 **dominates** x_2) iff $\forall i = 1, 2, \dots, m, f_i(x_1) \leq f_i(x_2)$ and it exists at least an index i_0 such that $f_{i_0}(x_1) < f_{i_0}(x_2)$
- $x_1 \sim x_2$ (x_1 and x_2 are **indifferent**) if x_1 does not dominate x_2 nor x_2 does not dominate x_1

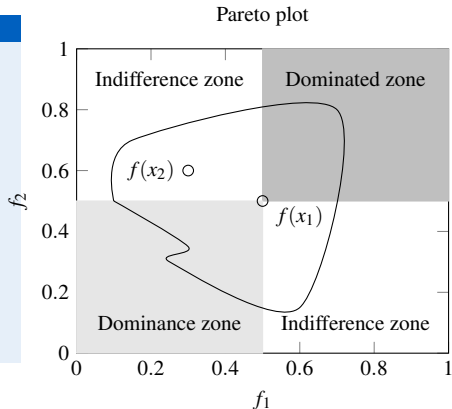


Pareto notion

Definition: Pareto dominance

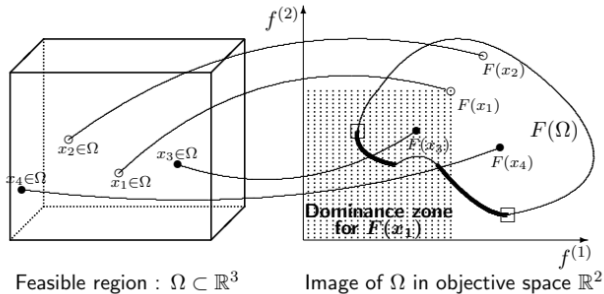
Given two decision vectors x_1 and x_2 in the feasible decision set, it is said that:

- $x_1 \leq x_2$ (x_1 **weakly dominates** x_2) iff $\forall i = 1, 2, \dots, m, f_i(x_1) \leq f_i(x_2)$
- $x_1 < x_2$ (x_1 **dominates** x_2) iff $\forall i = 1, 2, \dots, m, f_i(x_1) \leq f_i(x_2)$ and it exists at least an index i_0 such that $f_{i_0}(x_1) < f_{i_0}(x_2)$
- $x_1 \sim x_2$ (x_1 and x_2 are **indifferent**) if x_1 does not dominate x_2 nor x_2 does not dominate x_1



Pareto notion

- A point $x^* \in \Omega$ is **Pareto optimal** iff there is no $x \in \Omega$ such that $x < x^*$
- The set of Pareto (optimal) solutions is the **Pareto set** $\Omega_{\mathcal{P}}$
- The image of $\Omega_{\mathcal{P}}$ under the mapping F defines the solution to the problem and is called the **Pareto front** $F_{\mathcal{P}} \subseteq \mathbb{R}^p$



DMulti-MADS algorithm

DMulti-MADS: main algorithm

- 1: Initialization
 $x_0 \in \Omega, \Delta^0 \in (0, \infty), \tau \in (0, 1) \cap \mathbb{Q}, w^+ \in \mathbb{N}$
 initialize the list of non dominated points $L_0 = \{(x_0, \Delta_0)\}$
 - 2: Selection of the current incumbent point
 select (x_k, Δ_k) element of L_k such that $(x_k, \Delta_k) = \text{selectCurrentIncumbent}(L_k, w^+, \tau)$ (see Algorithm-a)
 set $\delta_k = \min\{\Delta_k, (\Delta_k)^2\}$, initialize $L_{add} := \emptyset$
 - 3: “Optional” global [Search](#) on mesh M^k
 evaluate f_Ω on a finite set of sample points S_k on the mesh M_k .
 set $L_{add} := \{((x_k, \Delta_k)) : x \in S_k\}$
 if $t < x_k$ for some $t \in S_k$, declare the iteration as successful and go to Step 5
 - 4: Local [Poll](#) on Mesh M^k
 select a positive spanning set D_Δ^k
 evaluate f_Ω at the set of poll points $P_k = \{x_k + \delta_k d : d \in D_\Delta^k\}$
 set $L_{add} := \{(x_k, \Delta_k) : x \in P_k\} \cup L_{add}$
 if $f(t) < f(x^k)$ for some $t \in P^k = \{x^k + \delta_k d : d \in D_\Delta^k\}$ in a finite subset of the mesh M^k
 if $t < x_k$ for some $t \in P_k$, declare the iteration as successful and go to Step 5
 - 5: Parameter update
 remove all dominated points of L_{add} .
 call the procedure $L_{k+1} := \text{updateList}(L_k, L_{add}, \tau)$ (see Algorithm-b)
 if the iteration is unsuccessful, replace the poll center (x_k, Δ_k) by (x_k, Δ_{k+1}) with $\Delta_{k+1} < \Delta_k, \dots, \Delta_{k+1} := \tau \Delta_k$
-

DMulti-MADS algorithm: update iterate list

Algorithm a: $\text{updateList}(L_k, L_{add}, \tau)$

```
1: for  $j = 0, 1, 2, \dots, |L_{add}|$  do
2:   if there exists at least an element  $(x, \Delta) \in L_k$  such that  $x_j < x$  then
3:     set  $L_k := L_k \setminus \{(x, \Delta) \in L_k : x_j < x\} \cup (x_j, \tau^{-1} \Delta_j)$ 
4:   else if there exists  $i = 1, 2, \dots, m$  such that  $f_i(x_j) < \min_{x \in L_k} f_i(x)$  then
5:     set  $L_k := L_k \cup (x_j, \tau^{-1} \Delta_j)$ 
6:   else if  $x_j \sim$  for all  $(x, \Delta) \in L_k$  then
7:     set  $L_k := L_k \cup (x_j, \Delta_j)$ 
8:   end if
9: end for
10: return  $L_k$ 
```

DMulti-MADS algorithm: select incumbent

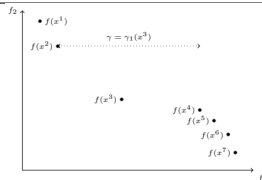
Algorithm b: selectCurrentIncumbent(L_k, w_+, τ)

```

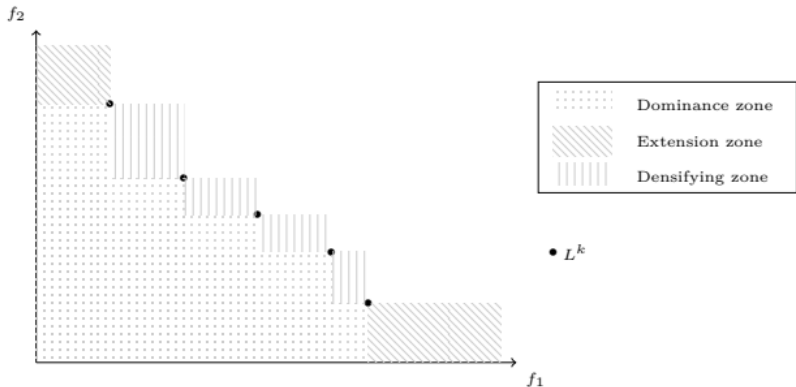
1: Let  $L_{select} := \{(x, \Delta) \in L_k : \tau^{w_+} \Delta_{\max_k} \leq \Delta \leq \Delta_{\max_k}\}$  with  $\Delta_{\max_k} = \max_{j=1,2,\dots,|L_k|} \Delta_j$ 
2: if  $|L_{select}| = 1$  then
3:   return  $(x, \Delta)$  with  $L_{select} = \{(x, \Delta)\}$ 
4: else if  $|L_{select}| = 2$  and  $|L_k| = 2$  then
5:   Let  $j_0 \in \arg \max_{j=1,2} \max_{i=1,2,\dots,m} f_i(x_j)$ 
6:   return  $(x_{j_0}, \Delta_{j_0})$ 
7: else
8:   Let  $j_0 \in \max_{j=1,2,\dots,|L_{select}|} \max_{i=1,2,\dots,m} \gamma_i(x_j)$ 
9:   return  $(x_{j_0}, \Delta_{j_0})$ 
10: end if

```

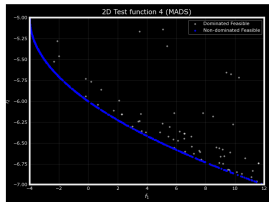
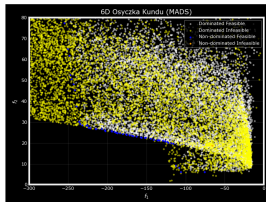
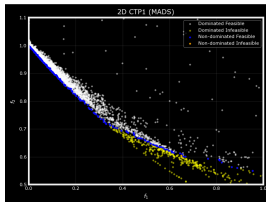
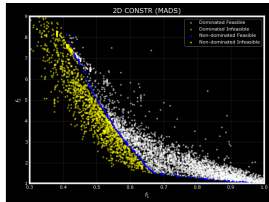
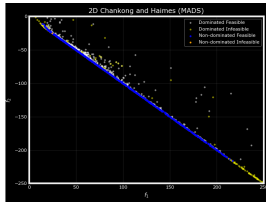
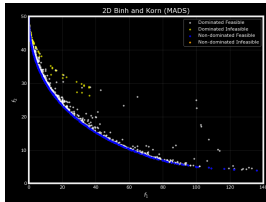
$$\gamma_i(x^j) = \begin{cases} 2 \frac{f_i(x^2) - f_i(x^1)}{f_i(x^{|L^k|}) - f_i(x^1)}, & \text{if } j = 1 \\ 2 \frac{f_i(x^{|L^k|}) - f_i(x^{|L^k|-1})}{f_i(x^{j+1}) - f_i(x^{j-1})}, & \text{if } j = 1 \\ \frac{f_i(x^{|L^k|}) - f_i(x^1)}{f_i(x^{|L^k|}) - f_i(x^1)}, & \text{otherwise} \end{cases}$$



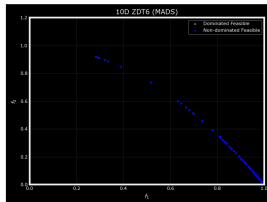
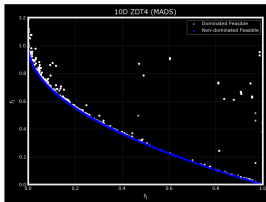
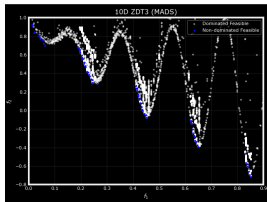
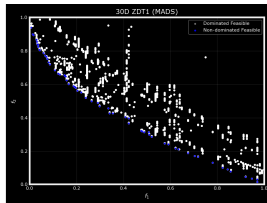
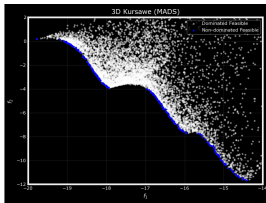
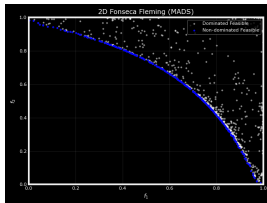
Zone of interests



Constrained MOP examples



Unconstrained MOP examples



- What is blackbox optimization?
- Functions differentiability
- Direct search methods
- Mesh adaptive direct search method
- Multiobjective optimization using MADS
- Implementations

MADS implementations (1/2)

- MADS is a general framework. It defines the conditions on the directions, but do not define the direction themselves
- There are several implementations:
 - **LT-MADS**: Based on Lower-Triangular random matrices
 - **QR-MADS**: Based on the QR decomposition and on normally distributed directions
 - **OrthoMADS**: Quasi-random, deterministic, and orthogonal directions. Current default in NOMAD

MADS implementations (2/2)

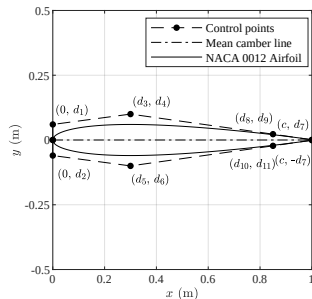
- Several programs that implement MADS are available:
 - **MATLAB:** NOMADm: https://github.com/khbalhandawi/MECH559_notebooks/blob/master/MATLAB_Algorithms/NomadM/nomadm.m
 - **Python:** OMADS: <https://ahmed-bayoumy.github.io/OMADS/>
 - **C++ (with interfaces to various languages):** NOMAD:
<https://nomad-4-user-guide.readthedocs.io/en/latest/index.html>
This implements the state-of-the-art in MADS development.

Transonic airfoil shape optimization

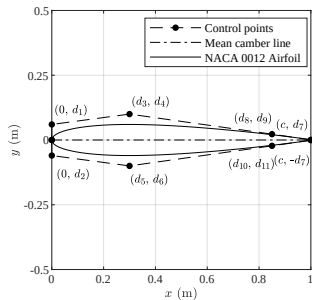
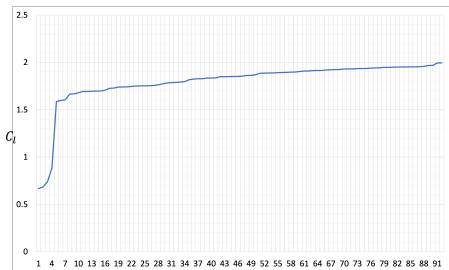
$$\min_{\mathbf{x} \in \Omega} -C_l(\mathbf{d})$$

$$\text{s.t.} \quad 0.075 - A_{cs}(\mathbf{d}) \leq 0,$$

$$C_d(\mathbf{d}) - 0.006 \leq 0$$



Transonic airfoil shape optimization



Transonic airfoil shape optimization

